

Київський національний університет імені Тараса Шевченка



ЗВІТ

до лабораторної роботи №4

Розширення можливостей PostgreSQL: користувацькі типи, функції та тригери

Виконала
Студентка 3 курсу
Групи МІТ-31
Факультету інформаційних технологій
Кубедінова Поліна

Київ, 2026

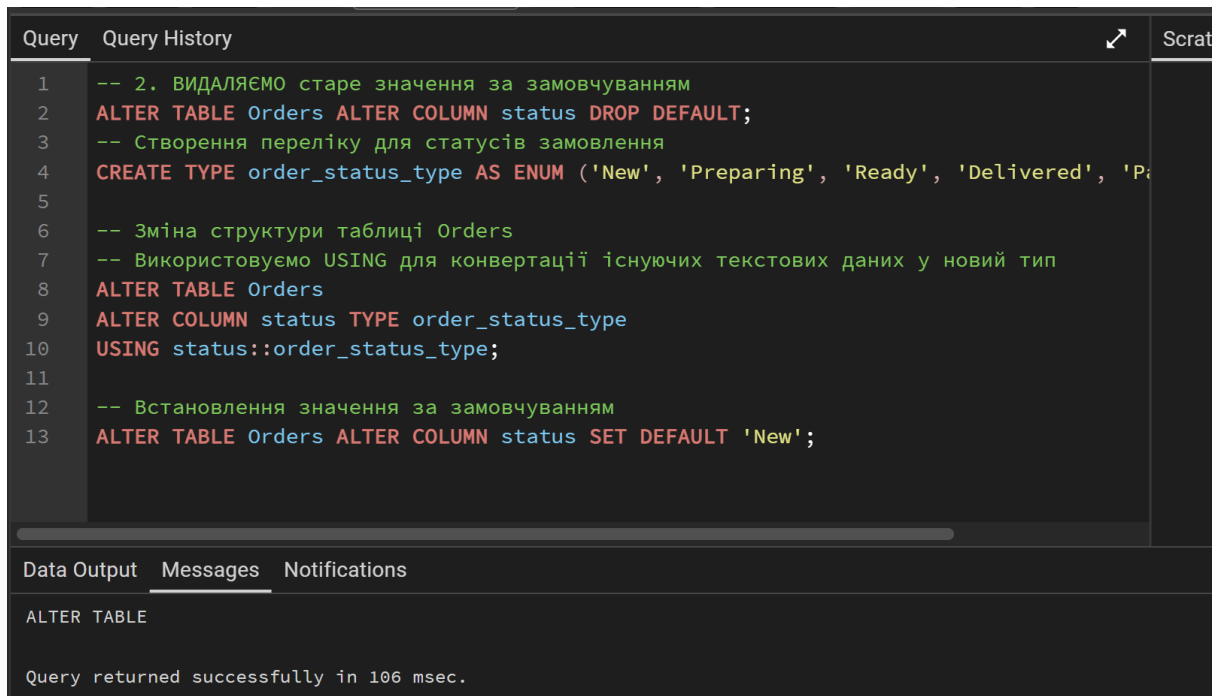
Тема: Розширення можливостей PostgreSQL: користувацькі типи, функції та тригери

Мета роботи: Закріпити знання з розширюваності PostgreSQL. Навчитися створювати користувацькі типи даних. Реалізувати власну користувацьку функцію або агрегат. Створити тригери для логування змін у базі даних. Автоматично оновлювати пов'язані таблиці чи заповнювати значення. Оновити діаграму бази даних відповідно до виконаних завдань. Перевірити коректність роботи реалізованих об'єктів через виконання тестових SQL-запитів.

Хід роботи

1. Створення користувацького типу ENUM

Замінімо текстове поле **status** у таблиці **Orders** на тип **ENUM**



```
Query  Query History  ↗  Scrat
1  -- 2. ВИДАЛЯЄМО старе значення за замовчуванням
2  ALTER TABLE Orders ALTER COLUMN status DROP DEFAULT;
3  -- Створення переліку для статусів замовлення
4  CREATE TYPE order_status_type AS ENUM ('New', 'Preparing', 'Ready', 'Delivered', 'Pa
5
6  -- Зміна структури таблиці Orders
7  -- Використовуємо USING для конвертації існуючих текстових даних у новий тип
8  ALTER TABLE Orders
9  ALTER COLUMN status TYPE order_status_type
10 USING status::order_status_type;
11
12 -- Встановлення значення за замовчуванням
13 ALTER TABLE Orders ALTER COLUMN status SET DEFAULT 'New';
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 106 msec.

2. Створення користувацької функції

Реалізуємо функцію, яка обчислює загальну вартість замовлення на основі даних з **Order_Items**. Це корисно для автоматизації розрахунків, які складно робити простим SELECT щоразу.

Query Query History

```

1 CREATE OR REPLACE FUNCTION calculate_order_total(p_order_id INT)
2 RETURNS DECIMAL(10,2) AS $$
3 DECLARE
4     v_total DECIMAL(10,2);
5 BEGIN
6     SELECT SUM(quantity * price_at_time)
7     INTO v_total
8     FROM Order_Items
9     WHERE order_id = p_order_id;
10
11     RETURN COALESCE(v_total, 0);
12 END;
13 $$ LANGUAGE plpgsql;
14
15 -- Тестовий запит для перевірки:
16 SELECT id, calculate_order_total(id) as real_sum FROM Orders;

```

Data Output Messages Notifications

Showing rows: 1 to 9

Page No: 1

	id [PK] integer	real_sum numeric
1	1	260.00
2	2	420.00
3	3	110.00
4	4	150.00
5	5	770.00

Total rows: 9 Query complete 00:00:00 2017

3. Створення тригерів для логування та автоматизації

Логування змін (Аудит)

Створимо таблицю для відстеження змін статусів замовлень, щоб знати, хто і коли змінив стан замовлення.

```

1 CREATE TABLE orders_audit_log (
2     log_id SERIAL PRIMARY KEY,
3     order_id INT,
4     old_status order_status_type,
5     new_status order_status_type,
6     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7     operation VARCHAR(10)
8 );
9
10 -- Функція тригера для логування
11 CREATE OR REPLACE FUNCTION log_order_status_change()
12 RETURNS TRIGGER AS $$
13 BEGIN
14     IF (TG_OP = 'UPDATE') THEN
15         IF (OLD.status <> NEW.status) THEN
16             INSERT INTO orders_audit_log (order_id, old_status, new_status, operation)
17             VALUES (OLD.id, OLD.status, NEW.status, 'UPDATE');
18         END IF;
19     ELSEIF (TG_OP = 'INSERT') THEN
20         INSERT INTO orders_audit_log (order_id, new_status, operation)
21         VALUES (NEW.id, NEW.status, 'INSERT');
22     END IF;
23     RETURN NEW;
24 END;
25 $$ LANGUAGE plpgsql;
26

```

```
25 $$ LANGUAGE plpgsql;
26
27 -- Прив'язка тригера
28 CREATE TRIGGER trg_audit_orders
29 AFTER INSERT OR UPDATE ON Orders
30 FOR EACH ROW EXECUTE FUNCTION log_order_status_change();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 94 msec.

Автоматичне оновлення суми замовлення

Створимо тригер, який автоматично перераховує **total_amount** у таблиці **Orders**, як тільки ми додаємо або змінюємо страву в **Order_Items**.

```
Query Query History
1 CREATE OR REPLACE FUNCTION update_order_total_amount()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Викликаємо раніше створену функцію для перерахунку
5     UPDATE Orders
6     SET total_amount = calculate_order_total(COALESCE(NEW.order_id, OLD.order_id))
7     WHERE id = COALESCE(NEW.order_id, OLD.order_id);
8
9     RETURN NULL;
10 END;
11 $$ LANGUAGE plpgsql;
12
13 -- Прив'язка тригера до деталей замовлення
14 CREATE TRIGGER trg_update_total
15 AFTER INSERT OR UPDATE OR DELETE ON Order_Items
16 FOR EACH ROW EXECUTE FUNCTION update_order_total_amount();
```

Data Output Messages Notifications

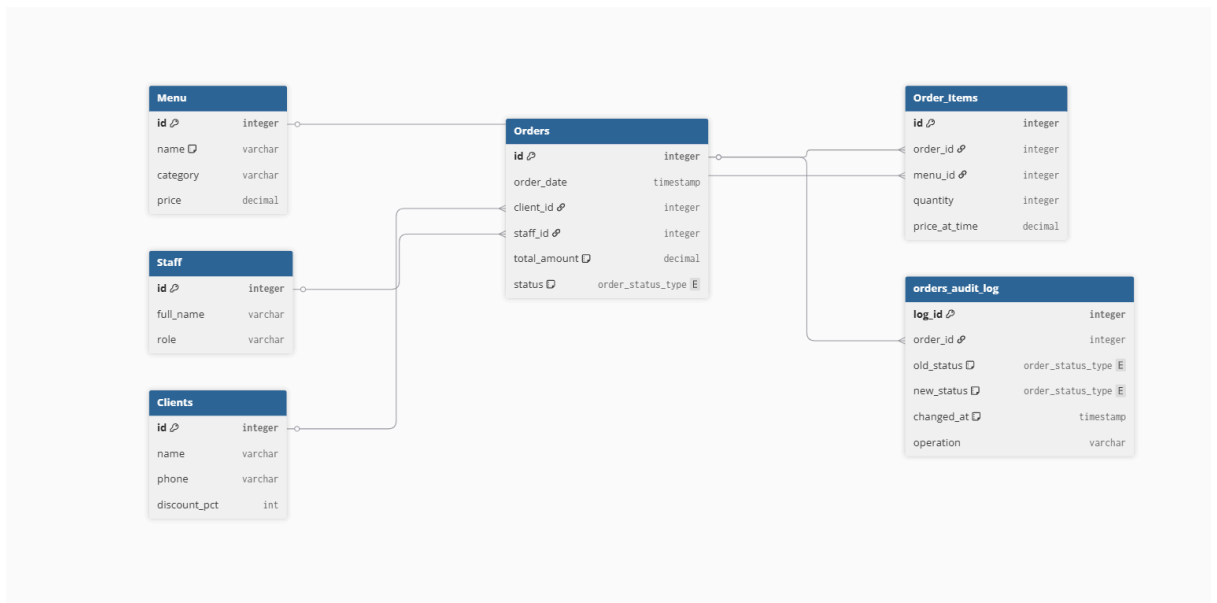
CREATE TRIGGER

Query returned successfully in 79 msec.

4. Оновлення діаграми бази даних

Що змінилося в діаграмі:

- Поле Status: Тепер воно використовує спеціальний тип **order_status_type** замість звичайного тексту.
- Таблиця **orders_audit_log**: З'явилася нова сутність, яка з'єднана з таблицею **Orders**. Це візуалізує ваш механізм відстеження змін.



5. Перевірка роботи (Тестові сценарії)

Тест тригера логування:

Query

Query History

Scratch

1

UPDATE Orders SET status = 'Ready' WHERE id = 1;

2

SELECT * FROM orders_audit_log;

Data Output

Messages

Notifications

</

Тест автоматичного підрахунку суми:

Query	Query History
1	<code>-- Додаємо нову страву в існуюче замовлення</code>
2	<code>INSERT INTO Order_Items (order_id, menu_id, quantity, price_at_time)</code>
3	<code>VALUES (1, 3, 2, 120.00);</code>
4	
5	<code>-- Перевіряємо, чи оновилась сума в таблиці Orders автоматично</code>
6	<code>SELECT id, total_amount FROM Orders WHERE id = 1;</code>

Data Output	Messages	Notifications						
Showing rows: 1 to 1 Page No: 1 <table> <tr> <th></th><th>id [PK] integer</th><th>total_amount numeric (10,2)</th></tr> <tr> <td>1</td><td>1</td><td>500.00</td></tr> </table>		id [PK] integer	total_amount numeric (10,2)	1	1	500.00		
	id [PK] integer	total_amount numeric (10,2)						
1	1	500.00						

Аналіз ефективності:

Користувацький тип: Зменшує ймовірність помилок ("paid" замість "Paid").

Тригери: Гарантують цілісність даних. Нам більше не потрібно вручну викликати UPDATE для суми замовлення після кожної проданої кави.

Аудит: Дозволяє адміністратору бачити історію роботи офіціантів та час виконання замовлень.

Висновок

У ході роботи було вдосконалено базу даних кафе шляхом впровадження типу **ENUM** для статусів замовлень, що забезпечило сувору цілісність даних. Реалізація користувацької функції та тригерів дозволила автоматизувати розрахунок вартості страв і виключити людські помилки при оновленні чеків. Створення системи аудиту в таблиці `orders_audit_log` забезпечило прозорий контроль за змінами статусів та діями персоналу. Оновлена ER-діаграма наочно відобразила нові архітектурні рішення, підтверджуючи готовність системи до масштабування. Виконання завдань закріпило навички програмування на PL/pgSQL та проектування логічно завершених інформаційних систем.